
Parte 2: Infraestructura y Precarga del Catálogo

1. El Concepto: "Data Source"

En una arquitectura limpia, la UI no debería saber dónde vive el archivo de la base de datos. Por ello, crearemos un servicio que se encargue de abrir Isar y, si el catálogo está vacío, lo llene con productos iniciales.

2. Implementación: Servicio de Base de Datos (`services/db_service.dart`)

Este servicio utiliza el patrón **Singleton** (a través de una variable estática) y se encarga de localizar la ruta física en el dispositivo móvil.

```
import 'package:isar/isar.dart';
import 'package:path_provider/path_provider.dart';
import '../models/producto.dart';
import '../models/carrito_item.dart';

class DBService {
  static late Isar isar;

  static Future<void> init() async {
    // 1. Obtener el directorio de documentos del sistema
    final dir = await getApplicationDocumentsDirectory();

    // 2. Abrir la base de datos con los esquemas generados
    isar = await Isar.open(
      [ProductoSchema, CarritoItemSchema],
      directory: dir.path,
    );

    // 3. Verificar si el catálogo necesita datos iniciales
    await _seedDatabase();
  }

  static Future<void> _seedDatabase() async {
    // Si ya existen productos, no hacemos nada
    final count = await isar.productos.count();
    if (count > 0) return;

    // Productos iniciales para nuestra tienda
    final catalogo = [
      Producto()..nombre = 'Monitor 4K'..precio = 350.0..categoria = 'Hardware',
      Producto()..nombre = 'Teclado Mecánico'..precio = 85.0..categoria =
'Periféricos',
      Producto()..nombre = 'Mouse Gamer'..precio = 45.0..categoria =
'Periféricos',
      Producto()..nombre = 'Laptop i7'..precio = 1200.0..categoria = 'Equipos',
    ];
```

```
// Escritura en bloque (Batch Write) para eficiencia
await isar.writeTxn(() async {
  await isar.productos.putAll(catalogo);
});
}
```

3. Integración en el punto de entrada (`main.dart`)

Como la inicialización de la base de datos es asíncrona, debemos asegurarnos de que termine antes de que Flutter intente dibujar la pantalla.

```
void main() async {
  // Asegura que los bindings de Flutter estén listos
  WidgetsFlutterBinding.ensureInitialized();

  // Inicializa Isar y el Catálogo
  await DBService.init();

  runApp(
    // ProviderScope es obligatorio para que Riverpod funcione
    const ProviderScope(child: MainApp()),
  );
}
```

4. Teoría detrás del código

- **`WidgetsFlutterBinding.ensureInitialized()`**: En ingeniería de Flutter, esta línea es obligatoria cuando tu `main` tiene código asíncrono (`await`) antes del `runApp`. Sin ella, la app podría colapsar al intentar acceder a servicios nativos (como el sistema de archivos).
- **Idempotencia del Seed**: El método `_seedDatabase` es **idempotente**; esto significa que aunque se llame mil veces, solo insertará los datos la primera vez (gracias al chequeo `count > 0`).
- **Write Transaction (`writeTxn`)**: En Isar, cualquier operación que modifique datos (insertar, borrar, actualizar) **debe** estar dentro de una transacción. Esto protege la integridad de los datos si la app se cierra inesperadamente durante la escritura.

5. Análisis

"Si en lugar de `DBService.init()` en el `main`, abriéramos la base de datos directamente en el `Widget`, ¿qué pasaría si el usuario navega rápidamente entre pantallas?"

(Respuesta: Se intentarían abrir múltiples conexiones al mismo archivo, lo que causaría un bloqueo o corrupción de la base de datos).
